# Lab 2: Threads Programming

Benjamin Maier, Francesco Astolfi

December 15 , 2025

All the code is been executed in google colab with the hosted GPU

# 1 Odd-even sort

## 1.1 Task 1

The initial parallelization of the Odd-Even Transposition Sort on the GPU involves executing the comparison and swap steps for each phase (odd or even) concurrently across multiple threads but in the same memory block.

- Synchronization Constraint: The implementation originally used a single thread block due to the constraint of the use __syncthreads() intrinsic. This function only provides a local barrier, guaranteeing synchronization within the current thread block only. This severely limits the scalability for sorting large arrays. For a complete sort on a large array, the entire array must be ordered and synchronized between more thread possibly part of other thread-blocks.
- Thread Block Size: Modern GPUs (Compute Capability 3.x and newer) commonly support up to 1024 threads per block. The use of a smaller block size, such as 512, unnecessarily underutilizes the hardware resources of a Streaming Multiprocessor (SM), potentially impacting overall occupancy and performance. Choosing instead a size over 1024 could bring errors out of resources, also if the big blocks are splitted in warps all the warps of the same block have to be executed in the same block.

**Performance impact:**

- Coalesced Memory Access: The parallel assignment of comparison pairs naturally leads to coalesced global memory access. This is because adjacent threads in a warp access consecutive, aligned memory locations.
- Control Divergence: The simple comparison and swap structure (if (d_data[j] > d_data[j + 1])) minimizes control divergence within a warp, as most threads are likely to follow the same execution path (either swap or no-swap), maintaining high instruction throughput.
- Shared Memory Experience: Experiments with tiling and explicit use of shared memory often show no significant improvement for this algorithm because the primary bottleneck is the large number of global memory reads/writes required by the sorting algorithm, not the latency of individual accesses, which are already well-hidden by coalescing. The vector of the number to order input could have a big size that doesn't fit in the shared memory
- Thread Coarsening : Each thread is assigned work using a stride loop structure (pair_id += nthreads). This mechanism ensures that the work of comparing multiple index pairs in the array is uniformly distributed among the available threads in the grid, maximizing utilization.

## 1.2 Measurements

| Run | Oddevensort-seq | Oddevensort-par[512] | Oddevensort-par[1024] |
|---|---|---|---|
| 1 | 342.985 s | 59.343 s | 45.662 s |
| 2 | 338.155 s | 59.552 s | 45.665 s |
| 3 | 338.366 s | 59.522 s | 45.655 s |
| 4 | 338.700 s | 59.549 s | 45.713 s |
| 5 | 336.863 s | 59.581 s | 45.853 s |
| AVG | 339.013 s | 59.509 s | 45.700 s |
| Speedup | 1.0 | 5.6968 | 7.4182 |

Table 1: Execution times and speedup for sequential and parallel odd-even sort size vector= $2^{19}$

In this implementation the conclusion is that the more resources are allocated the more speedup the system get. The upper bound of the speedup are the maximum thread that is physically available in a single thread-block.

```
./oddevensort_par
```

## 1.3 Task 2

This version implements an improved parallelization strategy for the Odd-Even Transposition Sort that overcomes the severe scalability limitations imposed by the single-block constraint of the __syncthreads() intrinsic used in Task 1. The new approach removes the __syncthreads() constraint by utilizing a multi-kernel launch pattern, with a fixed number of thread and blocks, which enables global synchronization across the entire GPU grid.

The key change is replacing the in-kernel loop over sorting phases with multiple, sequential kernel launches executed from the Host (CPU), this is because in each phase in the CPU before lunching the new kernel for the new phases the kernel already lunched is waited.

Host-Side Synchronization: The constraint on __syncthreads() is bypassed by launching a separate kernel for each phase (odd or even).

**Performance Impact:**

The shift to the multi-kernel approach fundamentally changes the thread lifecycle and introduces a new overhead compared to the single-kernel design of Task 1:

- Thread Lifespan Overhead: In Task 1, all threads within the single block were launched once and remained "alive" for the entire sorting process, spending time waiting at the syncthreads() barrier between phases. In the Multi-Kernel Approach (Task 2), the entire grid of threads is created and destroyed (or terminated/re-launched) for every phase of the sort. This repeated kernel launch and teardown introduces significant host-device communication overhead and GPU kernel scheduling overhead which is generally detrimental to performance compared to keeping threads resident.
- Scalability: The primary performance gain is scalability. The synchronization between the phases in the CPU allows the kernel to be launched with multiple thread, effectively utilizing the full compute capacity of the GPU's Streaming Multiprocessors (SMs) for large arrays. This resolves the limitation of the single-block structure in Task 1.

All other performance optimization logic mentioned in Task 1—including coalesced memory access, minimized control divergence, and efficient stride work distribution—remains valid and is carried over to this multi-kernel implementation.

## 1.4 Measurements

| Run | Oddevensort-seq | Oddevensort-par-multikernel[ t=512, b=16] | Oddevensort-par-multikernel[ t=1024, b=16] |
|---|---|---|---|
| 1 | 342.985 s | 5.103 s | 4.470 s |
| 2 | 338.155 s | 5.162 s | 4.443 s |
| 3 | 338.366 s | 5.121 s | 4.451 s |
| 4 | 338.700 s | 5.177 s | 4.437 s |
| 5 | 336.863 s | 5.175 s | 4.418 s |
| AVG | 339.013 s | 5.148 s | 4.444 s |
| Speedup | 1 | 65.8509 | 76.2818 |

Table 2: Execution times and speedup for sequential and parallel odd-even sort size vector= $2^{19}$

This version is largely better from the single block version; it utilize better the GPU resources.

The multikernel version with the more thread for blocks it got a 10% speedup so the achievable conclusion is that the physical GPU supports this amount of request and the work is more parallelized, there is an improvement in the memory occupancy.

```
./oddevensort_par_multikernel
```

# 2 Gaussian Elimination

## 2.1 Implementation

This CUDA implementation of the Gauss-Jordan row reduction algorithm leverages parallel processing on a GPU to solve a system of linear equations, where the solution vector is represented by Y. The process is highly parallelized, moving through the matrix one pivot row at time.

The main computation happens inside the work function's loop. Within each iteration, the operation is broken down into five distinct, synchronized phases executed on the GPU. The host-side cudaDeviceSynchronize() calls ensure that each phase completes entirely before the next one begins, enforcing the necessary data dependencies.

Before the loop begins, the input matrix and vectors and are copied from the host CPU memory to the device GPU memory (d_A, d_b, d_y). The results are copied back to the host after the main loop is completed. The copy of the value is row by row this for ensure that also with the size of matrix smaller than the max_size this not going to include the 0 value because the matrix A is initialized to the Max Size but then we use just the size*size values.

**The Five Parallel Phases (Per Pivot Row)**

The code implements Gauss-Jordan elimination, converting the matrix into the Identity Matrix while simultaneously transforming into the solution.

**1. Standardization of the Pivot Row ()**

- Kernel: scale_pivot_row
- Purpose: The pivot row is normalized so that the pivot element becomes 1.
- Parallelism: A 1D grid is used, with one thread per column element. A single designated thread (tid == 0) handles the calculation.

**2. Elimination of Elements Below the Pivot**

- Kernel: eliminate_rows
- Purpose: This performs the forward elimination step to zero out the elements and update the corresponding elements in the trailing submatrix.
- Parallelism: A grid (NUM_PAR_BLOCK_2D dimensions) is launched, with each thread processing a single element in the lower-right submatrix. This maximizes parallelism for the matrix updates.

**3. Update of Vector and Zeroing Out Column**

- Kernel: update_rows
- Purpose: Updates the elements of the right-hand side vector and sets the eliminated column elements to zero.
- Parallelism: A 1D grid is used, with one thread per row.

**4. Elimination of Elements Above the Pivot**

- Kernel: eliminate_rows_above
- Purpose: This is the Gauss-Jordan back elimination step, which ensures becomes the Identity Matrix. Its zeros out the elements and updates the upper-right submatrix.
- Parallelism: A 2Dgrid is launched, with each thread processing a single element in the upper-right submatrix.

**5. Update of Solution Vector and Final Zeroing of Column**

- Kernel: update_rows_above
- Purpose: Updates the previously calculated elements of the solution vector and sets the final eliminated column elements to zero.
- Parallelism: A 1D grid is used, with one thread per row.

**Performance impact:**
The design implements a fine-grained, element-wise parallel decomposition, which is a common and often effective strategy for dense linear algebra on GPUs.

- High Parallelism/Low Granularity: By launching a thread for nearly every element update (Phases 2 and 4 use a 2D grid where each thread handles one update), the design maximizes the number of concurrent operations. However, this results in low work granularity per thread, meaning each thread performs minimal computation, potentially leading to high overhead relative to work done.
- Memory Access: Matrix is stored in row-major order.
    - In the kernels (Phase 1, 3, 5), adjacent threads access adjacent columns/rows, which is favorable for coalesced memory access if the memory indices map correctly (e.g., increments by 1 for adjacent threads, which is excellent).
    - In the kernels (eliminate_rows and eliminate_rows_above), threads in the - dimension iterate across columns, accessing where increases, promoting good coalescing along the row.
- Divergence: There's no significant thread divergence within the kernels. The control flow is uniform across threads in each kernel, primarily consisting of boundary checks (if (j < N), if (row < N && col < N)), which are unlikely to cause significant warp divergence except at the edges of the matrix.
- Synchronization Overhead: The repeated use of cudaDeviceSynchronize() on the host between every phase of every pivot iteration (synchronizations total) is a major performance bottleneck. This forces the CPU to wait for the GPU to finish before launching the next small kernel.

| Run | Gaussianelimination-seq | Gaussianelimination-par [t1d=512,t2d=8x8] | Gaussianelimination-par [t1d=1024,t2d=16x16] |
|---|---|---|---|
| 1 | 3.794 s | 0.601 s | 0.572 s |
| 2 | 4.064 s | 0.589 s | 0.587 s |
| 3 | 4.165 s | 0.622 s | 0.596 s |
| 4 | 4.478 s | 0.592 s | 0.604 s |
| 5 | 4.388 s | 0.648 s | 0.582 s |
| AVG | 4.177 s | 0.610 s | 0.588 s |
| Speedup | 1 | 6.84 | 7.10 |

Table 3: Execution times and speedup for sequential and parallel gaussian Jordan elimination size matrix 2048x2048

When the configuration is scaled up to 1024 threads, the system is already constrained by the severe kernel launch overhead (Factor 1). Additionally, a block size of 1024 may require more register, potentially reducing the number of concurrent blocks per SM (lower Occupancy) compared to the smaller thread block, thus negating any benefit of a larger grid.

```
time ./gaussianelimination-par
```